

Production RAG System for an Enterprise Low-Code Platform

Role: Senior Full-Stack & AI Engineer (design, build, deploy) · **Domain:** Developer tooling / RAG · **Stack:** .NET 10, ONNX Runtime, Python, Angular

Context

A large metadata-driven enterprise application framework (~15,700 source files, .NET + Angular) needed an in-product assistant that could answer "where/how does X work" questions across its own codebase and documentation — for both internal developers and end customers building apps on top of it. Off-the-shelf cloud RAG was a non-starter: source code could not leave the customer's machine, and per-query API cost had to be zero for on-prem deployments.

Problem

- Semantic search over **~8,900 code+docs chunks** with low latency, fully **on-premise**.
- Multilingual queries (Italian developers, English codebase).
- No recurring API cost; must run inside the existing .NET backend, no extra service to operate.
- Answer quality measurable and defensible, not "vibes".

What I built

- **Hybrid retrieval:** BM25 (lexical) + `bge-m3` dense embeddings, fused and reranked by a **cross-encoder** (`bge-reranker-v2-m3`).
- **Fine-tuned the reranker** with a LoRA adapter ($r=16$, 3.4M trainable params, ~0.6% of the model) on mined hard-negatives — lifting retrieval **hit@8 from 0.75 to 0.81** and **MRR to 0.66** on a 600+ case eval set, with a held-out test set to guard against overfitting (Goodhart).
- **In-process .NET satellite via ONNX Runtime (CUDA):** the whole index stays hot in RAM inside the main backend — cold init ~10s, then **150–400 ms/query**. No separate Python service in production.
- **Query translation layer** (IT→EN, NLLB-distilled, cached) chosen by ablation over 178 cases as the best-performing configuration.
- **Local-LLM cutover:** replaced the cloud LLM with a self-hosted model (qwen2.5-coder:32B via Ollama) for the chat layer — **zero API cost**, ~91% correct tool-routing, ~8s end-to-end, tolerant parser for models that don't honor `tool_choice`.
- **Live notifications:** bootstrap progress + WebSocket push to a notification bell, with polling fallback after finding `SqlDependency` unreliable in production.

Results

Metric	Before	After
Retrieval hit@8	0.75	0.81
Retrieval MRR	—	0.66
Query latency (warm)	—	150–400 ms
Per-query API cost	cloud \$	\$0 (on-prem)
LLM tool-routing accuracy	—	~91%

Why it matters to a client

Most "RAG" work stops at a demo wired to a cloud API. This is a **measured, on-premise, zero-recurring-cost** system: fine-tuned reranking, a reproducible evaluation harness with a held-out set, and a local-LLM path that removes vendor lock-in and per-token cost entirely — exactly the constraints regulated and on-prem customers actually have.

Tech

`.NET 10` · `ONNX Runtime (CUDA)` · `BM25` · `bge-m3` · `bge-reranker-v2-m3` + `LoRA/PEFT` · `Python` (`transformers`, `sentence-transformers`) · `Ollama` / `qwen2.5-coder` · `Angular` · `WebSocket`

Client/product name withheld for confidentiality. Architecture and metrics available to discuss under NDA.